

Frontend Scaffold

Frontend Scaffold

Overview

This document provides a complete frontend scaffold for the Agent Dashboard MVP. It includes folder structure, TypeScript/React starter files, RxJS event bus wiring, session-bound components, tab model, and runtime glue. This scaffold is designed to align with the /spec contracts and the backend scaffold.

Folder Structure

```
/frontend
  /api
    backendClient.ts
  /state
    eventBus.ts
    sessionStreams.ts
  /components
    /tabs
      TabBar.tsx
      Tab.tsx
    /session
      AgentInputForm.tsx
      AgentOutputViewer.tsx
      AgentLogViewer.tsx
      AgentControlPanel.tsx
      AgentStateIndicator.tsx
    /layout
      Sidebar.tsx
      Workspace.tsx
  /models
    Tab.ts
    SessionViewModel.ts
  App.tsx
  index.tsx
```

File: /api/backendClient.ts

```
export class BackendClient {
  constructor(private baseUrl: string) {}

  async createSession(agentId: string, config: any, inputDraft: any) {
    const res = await fetch(`${this.baseUrl}/sessions/create`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ agentId, config, inputDraft })
    });
    return res.json();
  }

  async runSession(sessionId: string, input: any, config: any) {
    const res = await fetch(`${this.baseUrl}/sessions/${sessionId}/run`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ input, config })
    });
    return res.json();
  }

  async stopSession(sessionId: string) {
    const res = await fetch(`${this.baseUrl}/sessions/${sessionId}/stop`, {
      method: "POST"
    });
    return res.json();
  }
}
```

File: /state/eventBus.ts

```
import { Subject, ReplaySubject } from "rxjs";
import { filter, shareReplay } from "rxjs/operators";

export const globalEvents$ = new Subject<any>();
const sessionStreams = new Map<string, ReplaySubject<any>>();

export function routeEvent(event: any) {
  globalEvents$.next(event);

  if (!sessionStreams.has(event.sessionId)) {
    sessionStreams.set(event.sessionId, new ReplaySubject(100));
  }

  sessionStreams.get(event.sessionId)!.next(event);
}

export function sessionEvents(sessionId: string) {
  if (!sessionStreams.has(sessionId)) {
    sessionStreams.set(sessionId, new ReplaySubject(100));
  }

  return sessionStreams.get(sessionId)!.pipe(shareReplay({ bufferSize: 100 }));
}
```

File: /state/sessionStreams.ts

```
import { sessionEvents } from "../eventBus";
import { filter, scan } from "rxjs/operators";

export function logsForSession(sessionId: string) {
  return sessionEvents(sessionId).pipe(
    filter(e => e.type === "log.append"),
    scan((acc, e) => [...acc, e], [] as any[])
  );
}

export function partialOutputForSession(sessionId: string) {
  return sessionEvents(sessionId).pipe(
    filter(e => e.type === "output.partial"),
    scan((acc, e) => acc + e.chunk, "")
  );
}

export function finalOutputForSession(sessionId: string) {
  return sessionEvents(sessionId).pipe(
    filter(e => e.type === "output.final"),
    scan(_, e => e.output, null as any)
  );
}
```

File: /components/tabs/TabBar.tsx

```
import React from "react";
import { Tab } from "../../models/Tab";

export function TabBar({ tabs, activeTabId, onSelect, onClose, onNew }: {
  tabs: Tab[];
  activeTabId: string;
  onSelect: (id: string) => void;
  onClose: (id: string) => void;
  onNew: () => void;
}) {
  return (
    <div className="tab-bar">
      {tabs.map(tab => (
        <div
          key={tab.id}
          className={`tab ${tab.id === activeTabId ? "active" : ""}`}
          onClick={() => onSelect(tab.id)}
        >
          <span>{tab.title}</span>
          <span className="state-icon">{tab.stateIcon}</span>
          <button onClick={e => { e.stopPropagation(); onClose(tab.id); }}>×</button>
        </div>
      ))}
      <button onClick={onNew}>+</button>
    </div>
  );
}
```

File: /components/session/AgentInputForm.tsx

```
import React, { useState } from "react";
import { BackendClient } from "../../api/backendClient";

export function AgentInputForm({ sessionId, schema }: { sessionId: string; schema: any }) {
  const [form, setForm] = useState<any>({});
  const client = new BackendClient("http://localhost:3001");

  function updateField(key: string, value: any) {
    setForm({ ...form, [key]: value });
  }

  function run() {
    client.runSession(sessionId, form, {});
  }

  return (
    <div className="input-form">
      {Object.entries(schema.properties).map(([key, def]: any) => (
        <div key={key}>
          <label>{def.title}</label>
          <input
            value={form[key] || ""}
            onChange={e => updateField(key, e.target.value)}
          />
        </div>
      ))}
      <button onClick={run}>Run</button>
    </div>
  );
}
```

File: /components/session/AgentOutputViewer.tsx

```
import React, { useEffect, useState } from "react";
import { partialOutputForSession, finalOutputForSession } from "../../state/sessionStreams";

export function AgentOutputViewer({ sessionId }: { sessionId: string }) {
  const [partial, setPartial] = useState<string>("");
  const [finalOutput, setFinalOutput] = useState<any>(null);

  useEffect(() => {
    const sub1 = partialOutputForSession(sessionId).subscribe(setPartial);
    const sub2 = finalOutputForSession(sessionId).subscribe(setFinalOutput);
    return () => { sub1.unsubscribe(); sub2.unsubscribe(); };
  }, [sessionId]);

  return (
    <div className="output-viewer">
      <h3>Output</h3>
      {finalOutput ? (
        <pre>{JSON.stringify(finalOutput, null, 2)}</pre>
      ) : (
        <pre>{partial}</pre>
      )}
    </div>
  );
}
```

File: /components/session/AgentLogViewer.tsx

```
import React, { useEffect, useState } from "react";
import { logsForSession } from "../../state/sessionStreams";

export function AgentLogViewer({ sessionId }: { sessionId: string }) {
  const [logs, setLogs] = useState<any[]>([]);

  useEffect(() => {
    const sub = logsForSession(sessionId).subscribe(setLogs);
    return () => sub.unsubscribe();
  }, [sessionId]);

  return (
    <div className="log-viewer">
      <h3>Logs</h3>
      <ul>
        {logs.map((log, i) => (
          <li key={i} className={log.severity}>{log.message}</li>
        ))}
      </ul>
    </div>
  );
}
```

File: /components/session/AgentControlPanel.tsx

```
import React from "react";
import { BackendClient } from "../../api/backendClient";

export function AgentControlPanel({ sessionId }: { sessionId: string }) {
  const client = new BackendClient("http://localhost:3001");

  return (
    <div className="control-panel">
      <button onClick={() => client.stopSession(sessionId)}>Stop</button>
      <button onClick={() => client.runSession(sessionId, {}, {})}>Restart</button>
    </div>
  );
}
```


File: /components/session/AgentStateIndicator.tsx

```
import React, { useEffect, useState } from "react";
import { sessionEvents } from "../../state/eventBus";

export function AgentStateIndicator({ sessionId }: { sessionId: string }) {
  const [state, setState] = useState<string>("idle");

  useEffect(() => {
    const sub = sessionEvents(sessionId)
      .pipe()
      .subscribe(event => {
        if (event.type === "session.stateChanged") {
          setState(event.to);
        }
      });

    return () => sub.unsubscribe();
  }, [sessionId]);

  return <div className={`state-indicator ${state}`}>{state}</div>;
}
```

File: /models/Tab.ts

```
export type Tab = {
  id: string;
  title: string;
  sessionId: string;
  stateIcon: string;
};
```

File: /models/SessionViewModel.ts

```
export type SessionViewModel = {
  sessionId: string;
  agentId: string;
  state: string;
};
```

File: App.tsx

```
import React, { useState } from "react";
import { TabBar } from "../components/tabs/TabBar";
import { Workspace } from "../components/layout/Workspace";
import { Tab } from "../models/Tab";

export default function App() {
  const [tabs, setTabs] = useState<Tab[]>([]);
  const [activeTabId, setActiveTabId] = useState<string>("");

  function newTab() {
    const id = Math.random().toString(36).slice(2);
    const tab: Tab = { id, title: "New Session", sessionId: id, stateIcon: "○" };
    setTabs([...tabs, tab]);
    setActiveTabId(id);
  }

  function closeTab(id: string) {
    setTabs(tabs.filter(t => t.id !== id));
    if (activeTabId === id && tabs.length > 1) {
      setActiveTabId(tabs[0].id);
    }
  }

  return (
    <div className="app">
      <TabBar
        tabs={tabs}
        activeTabId={activeTabId}
        onSelect={setActiveTabId}
        onClose={closeTab}
        onNew={newTab}
      />
      {activeTabId && <Workspace sessionId={activeTabId} />}
    </div>
  );
}
```

File: /components/layout/Workspace.tsx

```
import React from "react";
import { AgentInputForm } from "../session/AgentInputForm";
import { AgentOutputViewer } from "../session/AgentOutputViewer";
import { AgentLogViewer } from "../session/AgentLogViewer";
import { AgentControlPanel } from "../session/AgentControlPanel";
import { AgentStateIndicator } from "../session/AgentStateIndicator";

export function Workspace({ sessionId }: { sessionId: string }) {
  return (
    <div className="workspace">
      <AgentStateIndicator sessionId={sessionId} />
      <AgentInputForm sessionId={sessionId} schema={{ properties: {} }} />
      <AgentOutputViewer sessionId={sessionId} />
      <AgentLogViewer sessionId={sessionId} />
      <AgentControlPanel sessionId={sessionId} />
    </div>
  );
}
```

Summary

This frontend scaffold provides:

- Complete folder structure
- RxJS event bus
- Session-bound components
- Tab bar and workspace
- Backend client
- React application shell

It is fully aligned with the /spec contracts and ready for implementation.